



UFRJ



INSTITUTO DE
COMPUTAÇÃO
UFRJ

Programação de Computadores II

Métodos de Ordenação

Prof. Marcos Tomazzoli Leipnitz

Sumário

- Ordenação por comparação e troca
 - *Bubble Sort*
 - *Insertion Sort*
 - *Selection Sort*

Tipos de algoritmos

- Ordenação por comparação e troca:
 - **Bubble Sort** – método da bolha - os elementos vão sendo ordenados do maior para o menor
 - **Insertion Sort** – os elementos são ordenados a partir do início do vetor
 - **Selection Sort** – passando pelo vetor, seleciona o menor (ou o maior) valor para a primeira posição, depois o segundo menor (ou maior), até os dois últimos elementos
- Ordenação por Divisão e Conquista:
 - **Merge Sort** – divide o vetor em partes (subproblemas), resolvendo esses vários subproblemas e depois une as soluções dos subproblemas

Bubble Sort

- Método da *bolha*
- Os elementos vão sendo ordenados do maior para o menor

Bubble Sort - Algoritmo

- Percorra todo o arranjo tomando seus elementos adjacentes par a par
- Se os elementos no par estiverem ordenados, siga ao próximo par
- Senão, troque-os de lugar
- Repita a operação até que nenhuma troca possa ser feita no arranjo inteiro

Bubble Sort

- Implementação em C

```
void bolha(int v[], int tam) {  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

Bubble Sort

- Observe que não precisamos correr sempre o arranjo até o final
- Basta irmos até onde garantimos estar ordenado...
- Marcando esse final
- Sempre corremos de 0 a **ult**
- E a cada passada, decrementamos **ult**

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){
    int ult, i, aux;
    for (ult=tam-1; ult>0; ult--){
        for (i=0; i<ult; i++){
            if (v[i] > v[i+1]) {
                aux = v[i];
                v[i] = v[i+1];
                v[i+1] = aux; }
        }
    }
}
```

tam	ult	i	aux
5	?	?	?

v →	[0]	[1]	[2]	[3]	[4]
	9	8	4	6	3

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

tam	ult	i	aux
5	4	0	?

v →	[0]	[1]	[2]	[3]	[4]
	9	8	4	6	3

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

tam	ult	i	aux
5	4	0	9

v →	[0]	[1]	[2]	[3]	[4]
	8	9	4	6	3

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

tam	ult	i	aux
5	4	1	9

v →	[0]	[1]	[2]	[3]	[4]
	8	4	9	6	3

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

tam	ult	i	aux
5	4	2	9

v →	[0]	[1]	[2]	[3]	[4]
	8	4	6	9	3

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

tam	ult	i	aux
5	4	3	9

v →	[0]	[1]	[2]	[3]	[4]
	8	4	6	3	9

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

tam	ult	i	aux
5	3	4	9

v →	[0]	[1]	[2]	[3]	[4]
	8	4	6	3	9

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

tam	ult	i	aux
5	3	0	8

v →	[0]	[1]	[2]	[3]	[4]
	4	8	6	3	9

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

tam	ult	i	aux
5	3	1	8

v →	[0]	[1]	[2]	[3]	[4]
	4	6	8	3	9

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){
    int ult, i, aux;
    for (ult=tam-1; ult>0; ult--){
        for (i=0; i<ult; i++){
            if (v[i] > v[i+1]) {
                aux = v[i];
                v[i] = v[i+1];
                v[i+1] = aux; }
        }
    }
```

tam	ult	i	aux
5	3	2	8

v →	[0]	[1]	[2]	[3]	[4]
	4	6	3	8	9

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

tam	ult	i	aux
5	2	3	9

v →	[0]	[1]	[2]	[3]	[4]
	4	6	3	8	9

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

tam	ult	i	aux
5	2	0	4

v →	[0]	[1]	[2]	[3]	[4]
	4	6	3	8	9

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

tam	ult	i	aux
5	2	1	6

v →	[0]	[1]	[2]	[3]	[4]
	4	3	6	8	9

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

tam	ult	i	aux
5	1	2	6

v →	[0]	[1]	[2]	[3]	[4]
	4	3	6	8	9

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

tam	ult	i	aux
5	1	0	4

v →	[0]	[1]	[2]	[3]	[4]
	3	4	6	8	9

Bubble Sort - Exemplo de Execução

```
void bolha(int v[], int tam){  
    int ult, i, aux;  
    for (ult=tam-1; ult>0; ult--)  
        for (i=0; i<ult; i++)  
            if (v[i] > v[i+1]) {  
                aux = v[i];  
                v[i] = v[i+1];  
                v[i+1] = aux; }  
}
```

tam	ult	i	aux
5	0	1	6

v →	[0]	[1]	[2]	[3]	[4]
	3	4	6	8	9

Insertion Sort

- Os elementos são ordenados a partir do início do vetor

Insertion Sort - Algoritmo

- Percorreremos o arranjo e, a cada novo elemento:
 - Procuramos onde, à esquerda desse elemento, ele se encaixa
 - Abrimos espaço para o elemento lá, deslocando para a direita todos os elementos que estão entre essa posição e a original do elemento

Insertion Sort

- Ou seja, aumentamos a parte ordenada do arranjo em uma posição, inserindo um novo elemento na posição correta e deslocando os demais para a direita
- Semelhante ao modo como ordenamos cartas de baralho

Insertion Sort - Melhoria

- Para cada passo do vetor original, a partir da posição 1 até o final
 - Deslocar o arranjo ordenado para a direita (uma posição por vez) até encontrar local adequado para o valor a ser inserido
 - Inserir esse valor na posição correta
- Evita assim a necessidade de se buscar a posição de antemão

Insertion Sort

- Implementação em C

```
void insercao(int v[], int tam){
    int i, aux, j;
    for (i=1; i<tam; i++) {
        aux=v[i];
        j=i;
        while ((j>0)&&(aux<v[j-1])){
            v[j] = v[j-1];
            j--; }
        v[j] = aux; } }
```

Insertion Sort

- Então...
- Corremos todos os possíveis candidatos a inserção
 - Enquanto o candidato estiver fora de lugar, deslocamos os anteriores a ele, até achar o lugar certo

Insertion Sort - Exemplo de Execução

```
void insercao(int v[], int tam){
    int i, aux, j;
    for (i=1; i<tam; i++) {
        aux=v[i];
        j=i;
        while ((j>0)&&(aux<v[j-1])){
            v[j] = v[j-1];
            j--; }
        v[j] = aux; } }
```

tam	i	aux	j
5	?	?	?

v →	[0]	[1]	[2]	[3]	[4]
	9	8	4	6	3

Insertion Sort - Exemplo de Execução

```
void insercao(int v[], int tam){
    int i, aux, j;
    for (i=1; i<tam; i++) {
        aux=v[i];
        j=i;
        while ((j>0)&&(aux<v[j-1])){
            v[j] = v[j-1];
            j--; }
        v[j] = aux; } }
```

tam	i	aux	j
5	1	8	1

v	[0]	[1]	[2]	[3]	[4]
	9	8	4	6	3

Insertion Sort - Exemplo de Execução

```
void insercao(int v[], int tam){
    int i, aux, j;
    for (i=1; i<tam; i++) {
        aux=v[i];
        j=i;
        while ((j>0)&&(aux<v[j-1])){
            v[j] = v[j-1];
            j--; }
        v[j] = aux; } }
```

tam	i	aux	j
5	1	8	0

v →	[0]	[1]	[2]	[3]	[4]
	8	9	4	6	3

Insertion Sort - Exemplo de Execução

```
void insercao(int v[], int tam){  
    int i, aux, j;  
    for (i=1; i<tam; i++) {  
        aux=v[i];  
        j=i;  
        while ((j>0)&&(aux<v[j-1])){  
            v[j] = v[j-1];  
            j--;  
        }  
        v[j] = aux; } }
```

tam	i	aux	j
5	2	4	2

v →	[0]	[1]	[2]	[3]	[4]
	8	9	4	6	3

Insertion Sort - Exemplo de Execução

```
void insercao(int v[], int tam){  
    int i, aux, j;  
    for (i=1; i<tam; i++) {  
        aux=v[i];  
        j=i;  
        while ((j>0)&&(aux<v[j-1])){  
            v[j] = v[j-1];  
            j--; }  
        v[j] = aux; } }
```

tam	i	aux	j
5	2	4	0

v →	[0]	[1]	[2]	[3]	[4]
	4	8	9	6	3

Insertion Sort - Exemplo de Execução

```
void insercao(int v[], int tam){  
    int i, aux, j;  
    for (i=1; i<tam; i++) {  
        aux=v[i];  
        j=i;  
        while ((j>0)&&(aux<v[j-1])){  
            v[j] = v[j-1];  
            j--;  
        }  
        v[j] = aux; } }
```

tam	i	aux	j
5	3	6	3

v →	[0]	[1]	[2]	[3]	[4]
	4	8	9	6	3

Insertion Sort - Exemplo de Execução

```
void insercao(int v[], int tam){  
    int i, aux, j;  
    for (i=1; i<tam; i++) {  
        aux=v[i];  
        j=i;  
        while ((j>0)&&(aux<v[j-1])){  
            v[j] = v[j-1];  
            j--;  
        }  
        v[j] = aux; } }
```

tam	i	aux	j
5	3	6	1

v →	[0]	[1]	[2]	[3]	[4]
	4	6	8	9	3

Insertion Sort - Exemplo de Execução

```
void insercao(int v[], int tam){  
    int i, aux, j;  
    for (i=1; i<tam; i++) {  
        aux=v[i];  
        j=i;  
        while ((j>0)&&(aux<v[j-1])){  
            v[j] = v[j-1];  
            j--; }  
        v[j] = aux; } }
```

tam	i	aux	j
5	4	3	4

v →	[0]	[1]	[2]	[3]	[4]
	4	6	8	9	3

Insertion Sort - Exemplo de Execução

```
void insercao(int v[], int tam){
    int i, aux, j;
    for (i=1; i<tam; i++) {
        aux=v[i];
        j=i;
        while ((j>0)&&(aux<v[j-1])){
            v[j] = v[j-1];
            j--; }
        v[j] = aux; } }
```

tam	i	aux	j
5	4	3	0

v →	[0]	[1]	[2]	[3]	[4]
	3	4	6	8	9

Selection Sort

- Passando pelo vetor, seleciona o menor (ou o maior) valor para a primeira posição, depois o segundo menor (ou maior), até os dois últimos elementos

Selection Sort - Algoritmo

- Primeiro encontre o menor elemento do arranjo
- Então troque esse elemento de lugar com o que está na primeira posição
- Encontre o segundo menor do arranjo
- Troque com o da segunda posição
- E assim por diante, até chegar ao fim do arranjo

Selection Sort

- Implementação em C

```
void selecao(int v[], int tam) {  
    int i, p, aux, posMenor;  
    for (i=0; i<tam-1; i++) {  
        posMenor = i;  
        for (p=i+1; p<tam; p++)  
            if (v[p] < v[posMenor])  
                posMenor = p;  
        aux = v[i];  
        v[i] = v[posMenor];  
        v[posMenor] = aux; } }
```

Selection Sort

- Implementação em C

```
void selecao(int v[], int tam) {  
    int i, p, aux, posMenor;  
    for (i=0; i<tam-1; i++) {  
        posMenor = i;  
        for (p=i+1; p<tam; p++)  
            if (v[p] < v[posMenor])  
                posMenor = p;  
        aux = v[i];  
        v[i] = v[posMenor];  
        v[posMenor] = aux; } }
```

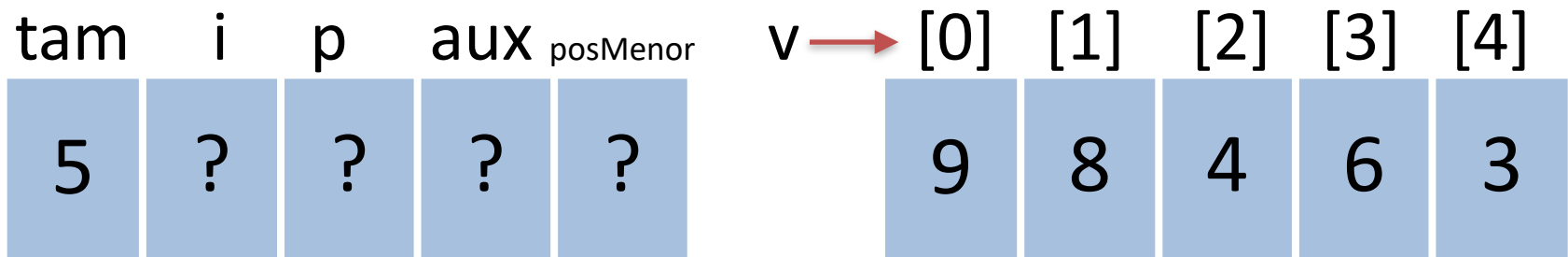
Busca o menor elemento

Selection Sort

- Para cada elemento do arranjo (exceto o último, que sobra já ordenado)
- Busca o menor elemento a partir deste
- Troca com a posição desse elemento

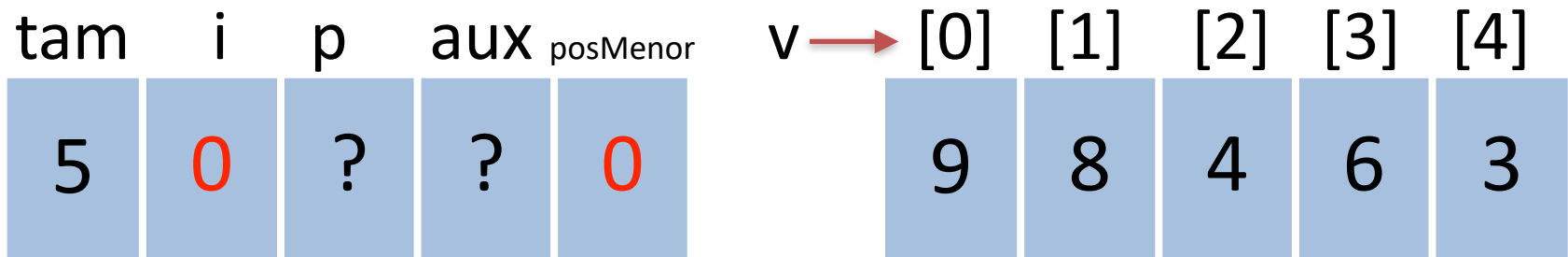
Selection Sort - Exemplo de Execução

```
void selecao(int v[], int tam) {  
    int i, p, aux, posMenor;  
    for (i=0; i<tam-1; i++) {  
        posMenor = i;  
        for (p=i+1; p<tam; p++)  
            if (v[p] < v[posMenor])  
                posMenor = p;  
        if (v[posMenor]<v[i]){  
            aux = v[i];  
            v[i] = v[posMenor];  
            v[posMenor] = aux; } } }
```



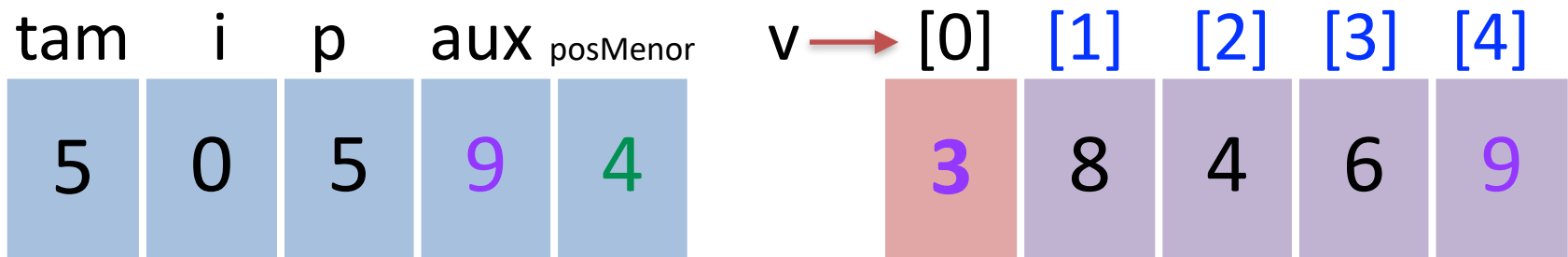
Selection Sort - Exemplo de Execução

```
void selecao(int v[], int tam) {  
    int i, p, aux, posMenor;  
    for (i=0; i<tam-1; i++) {  
        posMenor = i;  
        for (p=i+1; p<tam; p++)  
            if (v[p] < v[posMenor])  
                posMenor = p;  
        if (v[posMenor] < v[i]) {  
            aux = v[i];  
            v[i] = v[posMenor];  
            v[posMenor] = aux; } } }
```



Selection Sort - Exemplo de Execução

```
void selecao(int v[], int tam) {  
    int i, p, aux, posMenor;  
    for (i=0; i<tam-1; i++) {  
        posMenor = i;  
        for (p=i+1; p<tam; p++)  
            if (v[p] < v[posMenor])  
                posMenor = p;  
        if (v[posMenor]<v[i]){  
            aux = v[i];  
            v[i] = v[posMenor];  
            v[posMenor] = aux; } } }
```



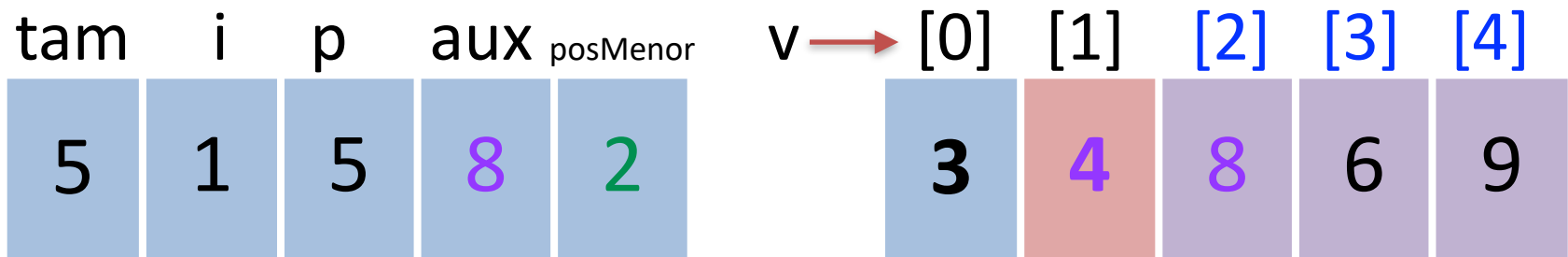
Selection Sort - Exemplo de Execução

```
void selecao(int v[], int tam) {  
    int i, p, aux, posMenor;  
    for (i=0; i<tam-1; i++) {  
        posMenor = i;  
        for (p=i+1; p<tam; p++)  
            if (v[p] < v[posMenor])  
                posMenor = p;  
        if (v[posMenor]<v[i]){  
            aux = v[i];  
            v[i] = v[posMenor];  
            v[posMenor] = aux; } } }
```

tam	i	p	aux	posMenor	v →	[0]	[1]	[2]	[3]	[4]
5	1	5	9	1		3	8	4	6	9

Selection Sort - Exemplo de Execução

```
void selecao(int v[], int tam) {  
    int i, p, aux, posMenor;  
    for (i=0; i<tam-1; i++) {  
        posMenor = i;  
        for (p=i+1; p<tam; p++)  
            if (v[p] < v[posMenor])  
                posMenor = p;  
        if (v[posMenor]<v[i]){  
            aux = v[i];  
            v[i] = v[posMenor];  
            v[posMenor] = aux; } } }
```



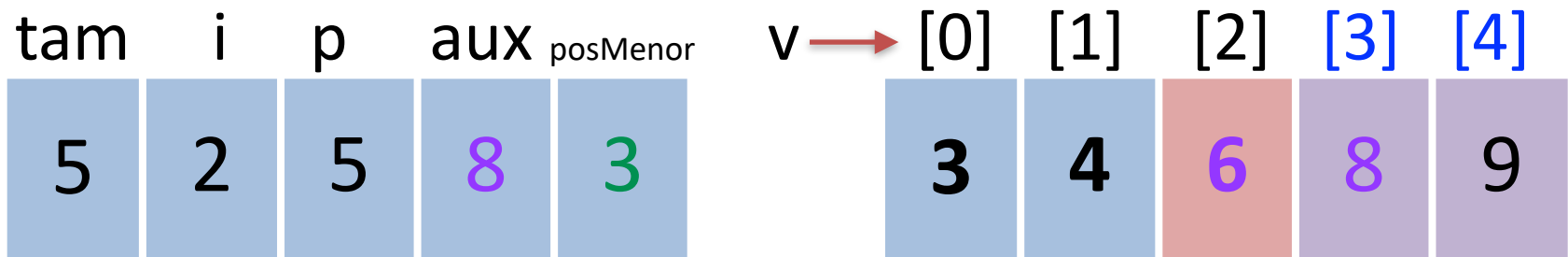
Selection Sort - Exemplo de Execução

```
void selecao(int v[], int tam) {  
    int i, p, aux, posMenor;  
    for (i=0; i<tam-1; i++) {  
        posMenor = i;  
        for (p=i+1; p<tam; p++)  
            if (v[p] < v[posMenor])  
                posMenor = p;  
        if (v[posMenor]<v[i]){  
            aux = v[i];  
            v[i] = v[posMenor];  
            v[posMenor] = aux; } } }
```

tam	i	p	aux	posMenor	v →	[0]	[1]	[2]	[3]	[4]
5	2	5	8	2		3	4	8	6	9

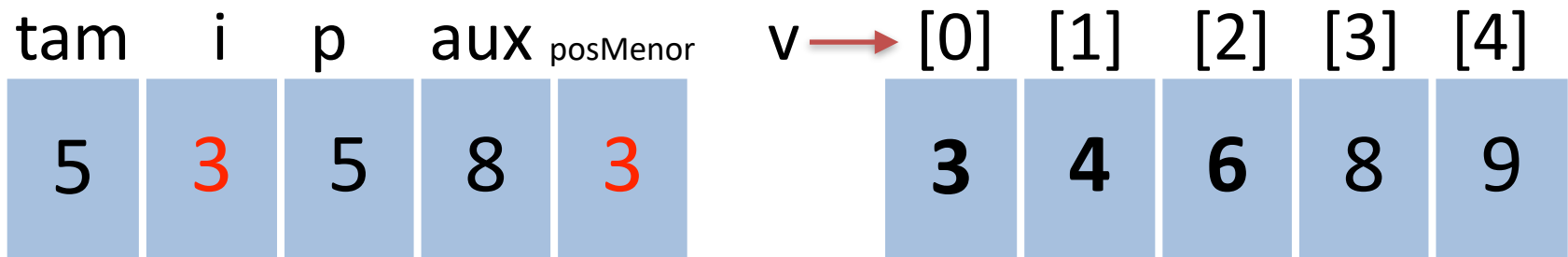
Selection Sort - Exemplo de Execução

```
void selecao(int v[], int tam) {  
    int i, p, aux, posMenor;  
    for (i=0; i<tam-1; i++) {  
        posMenor = i;  
        for (p=i+1; p<tam; p++)  
            if (v[p] < v[posMenor])  
                posMenor = p;  
        if (v[posMenor]<v[i]){  
            aux = v[i];  
            v[i] = v[posMenor];  
            v[posMenor] = aux; } } }
```



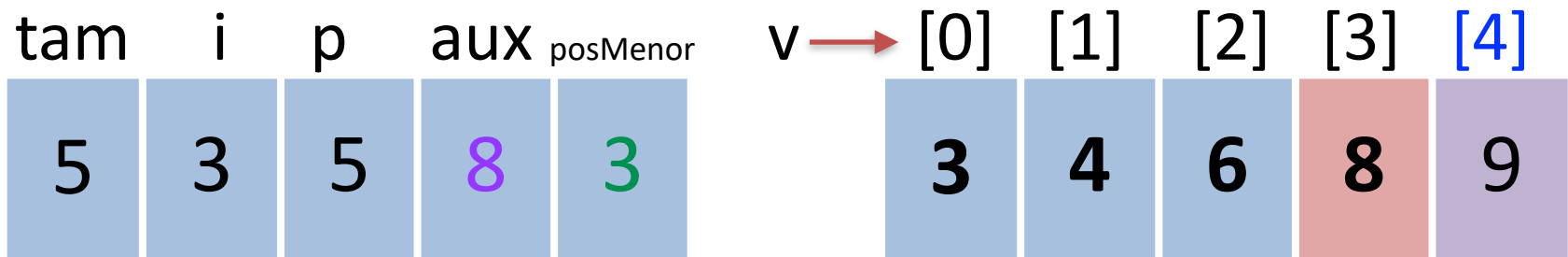
Selection Sort - Exemplo de Execução

```
void selecao(int v[], int tam) {  
    int i, p, aux, posMenor;  
    for (i=0; i<tam-1; i++) {  
        posMenor = i;  
        for (p=i+1; p<tam; p++)  
            if (v[p] < v[posMenor])  
                posMenor = p;  
        if (v[posMenor]<v[i]){  
            aux = v[i];  
            v[i] = v[posMenor];  
            v[posMenor] = aux; } } }
```



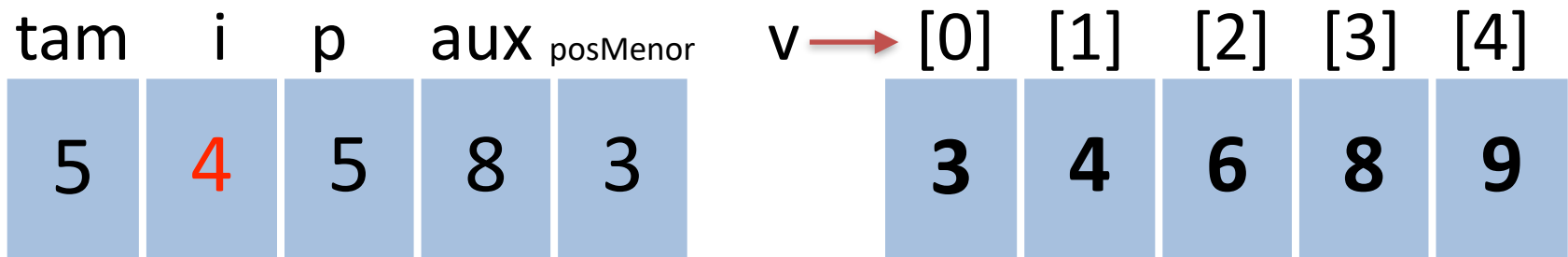
Selection Sort - Exemplo de Execução

```
void selecao(int v[], int tam) {  
    int i, p, aux, posMenor;  
    for (i=0; i<tam-1; I++) {  
        posMenor = i;  
        for (p=i+1; p<tam; p++)  
            if (v[p] < v[posMenor])  
                posMenor = p;  
        if (v[posMenor]<v[i]){  
            aux = v[i];  
            v[i] = v[posMenor];  
            v[posMenor] = aux; } } }
```



Selection Sort - Exemplo de Execução

```
void selecao(int v[], int tam) {  
    int i, p, aux, posMenor;  
    for (i=0; i<tam-1; i++) {  
        posMenor = i;  
        for (p=i+1; p<tam; p++)  
            if (v[p] < v[posMenor])  
                posMenor = p;  
        if (v[posMenor] < v[i]) {  
            aux = v[i];  
            v[i] = v[posMenor];  
            v[posMenor] = aux; } } }
```



Exemplos didáticos

- Bubble Sort:
 - <https://www.youtube.com/watch?v=lyZQPjUT5B4>
- Insertion Sort:
 - <https://www.youtube.com/watch?v=ROalU379l3U>
- Selection Sort:
 - <http://www.youtube.com/watch?v=Ns4TPTC8whw>
(versão levemente diferente do algoritmo)

Complexidade de algoritmos

- Comparativo entre algoritmos de ordenação (considerando Comparações)

Algoritmo	Comparações		
	Melhor	Médio	Pior
<i>Bubble Sort</i>	$O(n^2)$	$O(n^2)$	$O(n^2)$
<i>Insertion Sort</i>	$O(n)$	$O(n^2)$	$O(n^2)$
<i>Selection Sort</i>	$O(n^2)$	$O(n^2)$	$O(n^2)$

Análise de Complexidade - *Insertion Sort*

- Três cenários podem ser observados
 - **Melhor caso:** já ordenado crescentemente
 - Menor tempo de execução dentre as entradas de tamanho n
 - **Pior caso:** já ordenado decrescentemente
 - Maior tempo de execução dentre as entradas de tamanho n
 - **Caso médio:** não é, necessariamente, a média do melhor e do pior caso!
 - Média dos tempos de execução de todas as entradas de tamanho n

Complexidade de algoritmos

- Comparativo entre algoritmos de ordenação

Algoritmo	Comparações			Trocas		
	Melhor	Médio	Pior	Melhor	Médio	Pior
<i>Bubble Sort</i>	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n)^*$	$O(n^2)$	$O(n^2)$
<i>Insertion Sort</i>	$O(n)$	$O(n^2)$	$O(n^2)$	$O(n)^*$	$O(n^2)$	$O(n^2)$
<i>Selection Sort</i>	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n)^*$	$O(n)$	$O(n)$

*É possível adaptar a implementação para não fazer nenhuma troca.